

Staleness aware semi-asynchronous federated learning

Miri Yu, Jiheon Choi, Jaehyun Lee, Sangyoon Oh *

Department of Artificial Intelligence, Ajou University, Suwon, 16499, Republic of Korea

ARTICLE INFO

Keywords:

Federated learning
Semi-asynchronous
Staleness

ABSTRACT

As the attempts to distribute deep learning using personal data have increased, the importance of federated learning (FL) has also increased. Attempts have been made to overcome the core challenges of federated learning (i.e., statistical and system heterogeneity) using synchronous or asynchronous protocols. However, stragglers reduce training efficiency in terms of latency and accuracy in each protocol, respectively. To solve straggler issues, a semi-asynchronous protocol that combines the two protocols can be applied to FL; however, effectively handling the staleness of the local model is a difficult problem. We proposed SASAFL to solve the training inefficiency caused by staleness in semi-asynchronous FL. SASAFL enables stable training by considering the quality of the global model to synchronise the servers and clients. In addition, it achieves high accuracy and low latency by adjusting the number of participating clients in response to changes in global loss and immediately processing clients that did not participate in the previous round. An evaluation was conducted under various conditions to verify the effectiveness of the SASAFL. SASAFL achieved 19.69%p higher accuracy than the baseline, 2.32 times higher round-to-accuracy and 2.24 times higher latency-to-accuracy. Additionally, SASAFL always achieved target accuracy that the baseline can't reach.

1. Introduction

One crucial factor determining the performance of machine learning is the quantity and quality of data. In particular, to take advantage of exponentially growing data, distributed deep-learning techniques that use high-performance computing resources have rapidly advanced. Furthermore, numerous attempts have been made to perform distributed deep learning in federated learning (FL) environments, which offer benefits in terms of computational resource efficiency, training diverse data, and privacy.

FL leverages constrained computational resources and individual data generated from small-scale devices, such as IoT devices or mobile phones, to offer tailored services. The server responsible for updating the global model in FL does not directly participate in model training; rather, it aggregates local models received from client devices. Consequently, FL eliminates the need for resource intensive of high-performance computing infrastructure. Additionally, by encouraging increased client participation in model training, FL facilitates the integration of data from a more diverse distribution, thereby yielding a more intelligent model. In contrast to conventional distributed deep learning, where training data is collected at a centralized repository, FL

protect privacy by exclusively exchanging model parameters between participating servers and clients without sharing raw training data. This privacy-preserving mechanism stands contrast to traditional distributed deep learning frameworks, which are susceptible to privacy breaches due to the direct sharing of client-specific data.

Among the two types of FL, there are challenges in cross-device FL, in which hundreds of millions of small devices participate [1]. (1) **High communication cost:** The devices used by clients and servers in FL are typically connected through unstable networks, such as cellular networks or Wi-Fi. Thus, as the size of the training model increases, the process of uploading and downloading model requires more time. Unstable networks can also result in lost of the model during the transmission process. (2) **Statistical heterogeneity:** The FL training data were generated based on the usage patterns of different device users, resulting in training data that did not follow an independent and identical distribution (non-i.i.d.). This could result in either an increased training time owing to more frequent updates required for convergence or the risk of the divergence of the global model. (3) **System heterogeneity:** Because computation and communication resources are heterogeneous, training performance may be limited by slowest client, leading to delays in the training time. (4) **Privacy:** Even if training data is not shared, there

* Corresponding author.

E-mail addresses: zztiok@ajou.ac.kr (M. Yu), unidev@ajou.ac.kr (J. Choi), dlwogus8888@ajou.ac.kr (J. Lee), syoh@ajou.ac.kr (S. Oh).

<https://doi.org/10.1016/j.jpdc.2024.104950>

Received 19 December 2023; Received in revised form 1 May 2024; Accepted 24 June 2024

Available online 1 July 2024

0743-7315/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

is a possibility of personal information being leaked through the local model uploaded to the server. In addition, the training model may not converge because of model poisoning by an attacker.

Synchronous protocol can address the issues of statistical heterogeneity in FL. Attempts have also been made to apply asynchronous protocols to address the issue of delayed training speed owing to system heterogeneity in synchronous protocol. However, this further increases the communication cost imposed on the server and results the slow convergence of the global model. This is owing to the stale local model, which was trained based on an outdated global model. This problem is exacerbated when clients are slow because it cannot receive global model frequently. Recently, several studies have addressed the problem of poor training performance of slow clients in both protocols by applying semi-asynchronous protocols. This protocol can alleviate the problem of increased staleness from slow clients by synchronising some clients with the server, similar to synchronous FL, while allowing other clients to perform local updates independently, similar to asynchronous FL.

The semi-asynchronous protocol forcibly synchronises the stale local model with the latest global model when staleness exceeds a threshold. This was performed to resolve the performance degradation of the global model by improving the quality of the local model. However, if the quality of the latest global model is low, the quality of the synchronised local model will also be low. These problems have not been addressed in existing studies.

Increasing the number of participating clients in the aggregation can address the decline in accuracy caused by stale local models. There are two approaches for this. One is determining the deadline for the waiting time, allowing as many clients as possible to upload the local model. Alternatively, the number of participating clients must be determined. However, as more clients participated, the waiting time required per round increased. Therefore, having several participating clients is not always optimal. It is necessary to increase or decrease the number of participating clients adaptively based on the training progress of the global model.

The number of participating clients of the some previous semi-asynchronous method is fixed. Consequently, even if many clients upload their local models, some of them remain unnecessarily stale because they are unable to participate in the aggregation owing to fixed number of participating clients. Thus, strict constraints on the number of participating clients must be avoided.

In this study, we proposed staleness aware semi asynchronous federated learning (SASAFL), which aims to solve the problem of reduced training efficiency caused by staleness. SASAFL actively uses the global loss to increase the training efficiency of the global model. To improve the quality of the local model, it considers staleness of local model and global loss when synchronising the server and client. It also adjusts the number of participating clients, as required, based on changes in the global loss, to improve the accuracy of the global model with low latency. In addition, by setting the minimum number of participating clients, it is possible to allow more local models can be aggregated. This aids to eliminate unnecessary stale local models.

The contributions of SASAFL are as follows:

- Considering the training inefficiency problem caused by staleness in semi-asynchronous protocols, we proposed SASAFL, which improves the quality of local models and global models degraded owing to staleness problems.
- We built a multicore and multinode-based FL experimental environment to evaluate the SASAFL. Through our evaluation, we demonstrated that SASAFL provides stability in training by increasing the quality of the local model and enables efficient training in terms of accuracy and latency by increasing the quality of the global model.

2. Background and related work

FL trains a global model by sharing only the model between the server and client. After the server distributes the global model to all clients, each client performs local updates using its local dataset to train the received global model and create a local model. These local models are uploaded back to the server. The server subsequently replaces the global model to the results of aggregating the received local models. In this study, we defined a *round* as the cycle from distribution to updating the global model. Each index of the round corresponds to a *version* of the global model. Three protocols are applicable between the server and client in FL: synchronous, asynchronous, and semi-asynchronous.

Synchronous FL The server selects the clients to participate in each round. Once all selected clients upload their local models, the aggregation process starts updating the global model (Fig. 1-(a)). Because all selected clients begin local updates using the same global model, the global model can converge to a high accuracy within a few rounds. However, owing to system heterogeneity, which is one of the challenges of FL, the number of rounds is limited to the speed of the slowest client, also known as a *straggler*. This limitation leads to an increase in the training latency.

Several FL studies have assumed a synchronous protocol. FedAvg [2] randomly selected participating clients in every round. To reduce the latency caused by stragglers, [3] measured the computation and communication latencies of clients to select clients capable of uploading local models within the deadline of each round. [4] clustered clients with similar latency into tiers and randomly selected clients from one tier, aiming for a similar latency among participating clients in each round. [5,6] allowed different numbers of local epochs to adapt to the computational capabilities of each client. Conversely, to address statistical heterogeneity, [7,8] calculated the utility (i.e., contribution to the global model) of each client based on the local loss. This ensured that only clients with high utility participate in each round. In addition, [9] adjusted the local epoch for each client based on the local loss between two adjacent rounds.

Asynchronous FL In asynchronous FL, all clients perform local updates and receive the global model from the server independently. Therefore, there is no need to wait for other clients to complete the local updates. Whenever one client uploads a local model, the server updates the global model only using that. Subsequently, the server immediately sends the updated global model to that client (Fig. 1-(b)). Each client performs a local update from a different state (i.e., version) of a global model, called the *base global model*. In addition, as the number of clients increases, there is a significant difference between the states of the global and local model (i.e., the stale local model). This difference increases the *staleness* of the local model. Staleness refers to the difference of state between the global and local models. This can be calculated as the difference in version between the current and base global models of each client. In particular, stragglers received the global model less frequently than other clients, resulting in increased staleness and quality deterioration of local model. [10] applied a decay coefficient to local models that had tolerable staleness to solve the problem of slow convergence of the global model caused by the staleness of clients. [11] allows partial model training to solve the staleness of the straggler's local model, and has a similar approach to SASAFL, which will be explained later. However, it does not solve the problem of staleness caused by frequently updated global models due to many clients.

Semi-asynchronous FL The semi-asynchronous protocol can address the issues of increased training latency in synchronous FL and the problem of decreased training accuracy in the asynchronous FL owing to the straggler. In this protocol, some clients receive the latest global model by synchronising with the server, like synchronous FL. Meanwhile, other clients continue to perform local updates based on the outdated global model rather than the latest global model, similar to the asynchronous FL (Fig. 1-(c)). That is, the local models used for aggregation may or may not be trained based on the same global models.

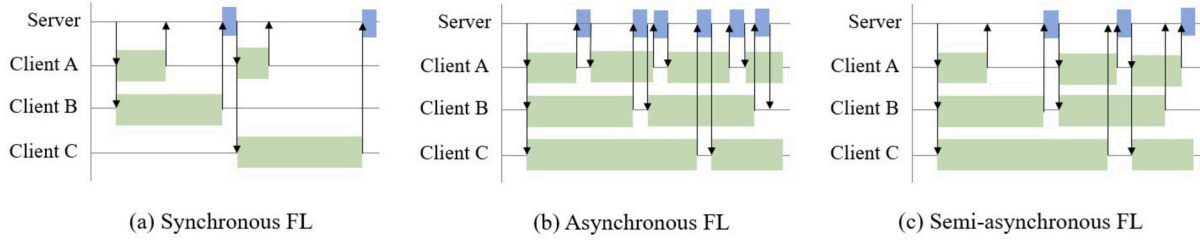


Fig. 1. Illustration of the synchronous, asynchronous and semi-asynchronous FL.

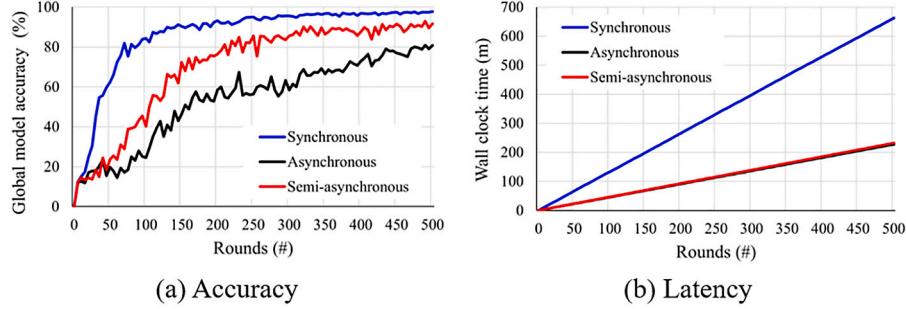


Fig. 2. Comparison of synchronous, asynchronous and semi-asynchronous FL. The left corresponds to accuracy, and the right corresponds to wall clock time.

Fig. 2 compares the accuracy and wall clock time of synchronous, asynchronous, and semi-asynchronous FL over 500 rounds using the MNIST [12] dataset. The number of rounds for asynchronous FL is measured as the number of the update for global model. Semi-asynchronous FL had a final accuracy of 91.6%, which was 6%p lower than synchronous FL (Fig. 2-(a)), but latency performance was approximately three times faster than synchronous FL (Fig. 2-(b)). Additionally, with the target accuracy set to 90%, the latency of semi-asynchronous FL was lower, taking approximately 143 min, while synchronous FL took approximately 177 min. Conversely, it achieved a 10%p higher final accuracy while maintaining the same latency as asynchronous FL. In semi-asynchronous FL, the server can receive local models during aggregation. These local models may impact on the next aggregation. Therefore, as training progresses in semi-asynchronous FL, the waiting time for the server to receive local models may become shorter or even nonexistent in some cases, resulting in latency almost the same as asynchronous FL. This experiment shows that the same global model in which participating clients perform local updates has a significant impact on accuracy, and that independent local updates of all clients have a significant impact on latency. In other words, semi-asynchronous FL is superior to synchronous FL in terms of latency and asynchronous FL in terms of accuracy.

However, because of the presence of stragglers, their staleness more increases. In addition, there is an issue with the deterioration of the global model training performance in terms of accuracy and latency due to stragglers. One possible approach for solving the staleness problem of semi-asynchronous protocols is to set a deadline for a round. This deadline encompasses the time receiving local models. In [13], static deadlines were set and the computing and communication latencies of clients were measured. Clients were clustered based on the latency of model training and uploading. All local model of clients in the clusters that could be uploaded within deadline were used for aggregation. In [14,15], the maximum deadline for each round was dynamically determined based on the maximum computation and communication latency of the clients. Another approach is to determine the number of clients participating in the aggregation. In [16], the time required for one round for each client was measured to determine the number of participating clients adaptively. [17] used deep reinforcement learning (DRL) to consider the latency of the clients. Additionally, [18] employed DRL to consider latency and accuracy simultaneously.

3. Motivation

To address the training inefficiency caused by staleness, a semi-asynchronous Federated Learning (FL) approach is employed, which means that both synchronous FL and asynchronous FL are utilized “partially”. [19] trains asynchronously, but changes to synchronous protocol when the model discrepancy defined by the EMD value exceeds the threshold. During this process, there’s an overhead involved in calculating the EMD value, and the challenge lies in the inability to mitigate the straggler problem that arises during the synchronous FL. [20] also has similar approach. After clustering based on processing time of each client, local model is updated synchronously inside the cluster, but cluster model is aggregated with global model asynchronously. This solution also has the limitations described above in terms of straggler. Additionally, clusters containing stragglers continue to have low contributions, so the global model cannot sufficiently train the training data of the cluster. However, the semi-asynchronous protocol which merged between synchronous and asynchronous FL as described in Section 2 might solve the inefficiency training problem owing to staleness.

In semi-asynchronous FL, the global model is updated by aggregating local models. This implies that the convergence of the global model is determined by the quality of the local model. In addition, because the base of local model is a global model, it is also important to consider which global model the client will initiate local updates. Therefore, also the base global model of the local model is related to FL process performance. A previous study [15] which emphasised the significance of high-quality local models for the convergence of the global model suggested forcibly synchronising the too-stale-local model with the latest global model. This approach enabled the ongoing training of the local model without being too stale.

[21] adjusts the importance of local models based on staleness and the impact of local loss to solve inefficient training due to staleness, and has an approach close to SASAFL, which will be explained later. They proposed a solution that uses reinforcement learning to determine the time slot that interpolates the local model and the fresh global model. However, because even the latest global model cannot guarantee quality, the quality of the local model cannot be always improved by synchronising it with the latest global model. If only some of the total clients frequently participate in aggregation, a global model may be overfitted to the local datasets of these clients. In this case, synchronisation with the latest global model may further deteriorate the quality of

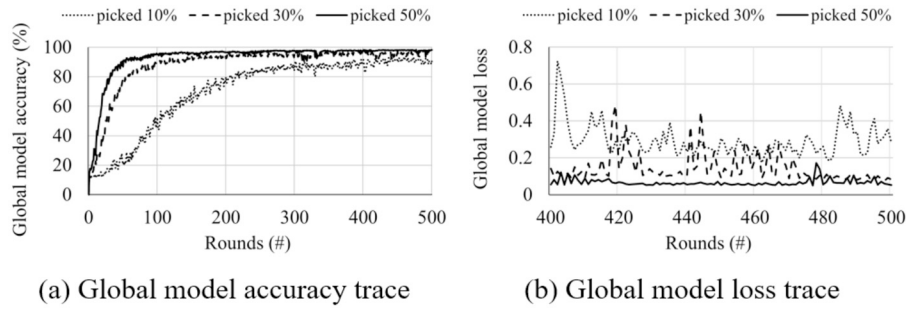


Fig. 3. Accuracy and loss of the global model when setting the ratio of participating clients in one round among 30 clients in the semi-asynchronous FL process to 10%, 30%, and 50%.

the local model. To address this issue, the SASAFL synchronises a local model with the latest global model considering not only the staleness of the local model and but also the *global loss*. Here, global loss refers to the loss value obtained by evaluating the global model using test dataset of the server. The SASAFL determines that the quality of a global model is better when it has a lower global loss.

Another important factor for improving the quality of the global model is the number of participating clients. In previous studies using the semi-asynchronous protocol, only some clients (i.e., clients participated in previous round or their staleness of local model are too large) are synchronized with the server while the rest perform local updates independently [13–16,18]. As the number of local models synchronised to the same state increases (i.e., the number of participating clients increases), it gets closer to the synchronous FL and can converge to high accuracy in fewer rounds. Fig. 3 shows the accuracy and loss when the ratio of the number of clients participating in the aggregation among the 30 clients in the semi-asynchronous FL process is set to 10%, 30%, and 50%. When comparing 10% and 50% participation, the more clients participate in the aggregation, the faster it converges to high accuracy (Fig. 3-(a)) and low loss (Fig. 3-(b)). However, the time required for each round increases.

To solve this issue, existing studies [14,15] measured the latency by profiling the client's resources (CPU, GPU, memory, network bandwidth, etc.) to determine the deadlines or number of participating clients in one round. However, the quality of the global model could be improved by considering not only the client's latency but also its accuracy or loss.

Another limitation of existing studies is that they do not consider that aggregation and local model reception can be performed in parallel. In the case of a method for adjusting the number of participating clients, if the determined number of clients participate, the local model reception is stopped and aggregation begins. If many clients participate in the FL process, many can upload local models for aggregation in a short period of time. However, they cannot be received in the current round, but only in the after. The number of these clients affects the fixed number of participating clients. It results that aggregation is frequently performed, further increasing the staleness of clients. Even if it is assumed that aggregation and local model reception are performed in parallel, only a strictly fixed number of clients is used for each aggregation, resulting in other clients becoming unnecessarily stale.

As shown in Fig. 4, we assume that the aggregation and local model reception are performed in parallel. Six clients uploaded their local model in round four. Among them, only clients A and D participated in aggregation, because the determined number of participating clients were two in that round. Consequently, the remaining clients (i.e., clients B, C, E, and F) cannot participate to aggregation of round four. When aggregation in round four is completed, there are four local models in the buffer. If the number of participating clients in round five is less than four, the client who fails to participate in round five cannot participate once again. For example, if the number of participating clients in round five is two, clients E and F, which were received last, cannot participate again, resulting in higher staleness. As explained in detail later, SASAFL

did not strictly enforce the number of participating clients in a round to solve this problem.

4. SASAFL: staleness aware semi asynchronous federated learning

Semi-asynchronous FL protocol allows stale local models to be aggregated, thereby reducing the training performance of the global models. To solve this problem, (1) clients in SASAFL decided whether to receive the global model based on its quality. This decision aimed at improving the quality of local model. (2) The SASAFL allowed adaptive aggregation by adjusting the number of clients participating in the aggregation based on changes in global loss. Through these two approaches, SASAFL enabled efficient semi-asynchronous FL while allowing the use of stale local models.

Fig. 4 shows an overview of the SASAFL with six clients. First, the server distributed the global model to all the clients. Each client then decided whether to receive the global model based on its reception policy. However, all clients initially received the global model, and then began to make local updates based on it. After distributing the global model, the server determined the minimum number of clients required to participate in the aggregation in next round. This was performed based on the policy of adjusting the number of participating clients. Once a local update was completed, the client uploaded the local model to the server. The server collected the local model in a buffer whenever it was received. Once the local model was received from the determined minimum number of clients, the aggregation for updating global model began. Assuming that the server could receive the local model during aggregation, it also collected the local models during aggregation. The collected local models were then used to aggregate in the next round. The updated global model was distributed to all the clients again, as before.

In this section, we introduce a policy for global model reception and adjusting the number of participating clients in the aggregation to explain the SASAFL details using Fig. 4.

4.1. Global model reception policy

Various types of clients In SASAFL, when a new round begins, the server distributes the global model to all clients. Therefore, all the clients decide whether to receive a global model in every round. Currently, the clients are classified into several types. First, the clients are divided into two categories: *waiting* and *active* clients. This categorization is based on whether a local model has been uploaded. Waiting clients include *up-to-date*, *tolerable*, and *undrafted* clients, defined as follows:

- **Up-to-date:** Clients who completed a local update based on the latest global model and then uploaded their local model to the server in the previous round. (e.g. clients C and D in round two as shown in Fig. 4).
- **Tolerable:** Clients who completed a local update based on the not-too-outdated global model and then uploaded the local model to

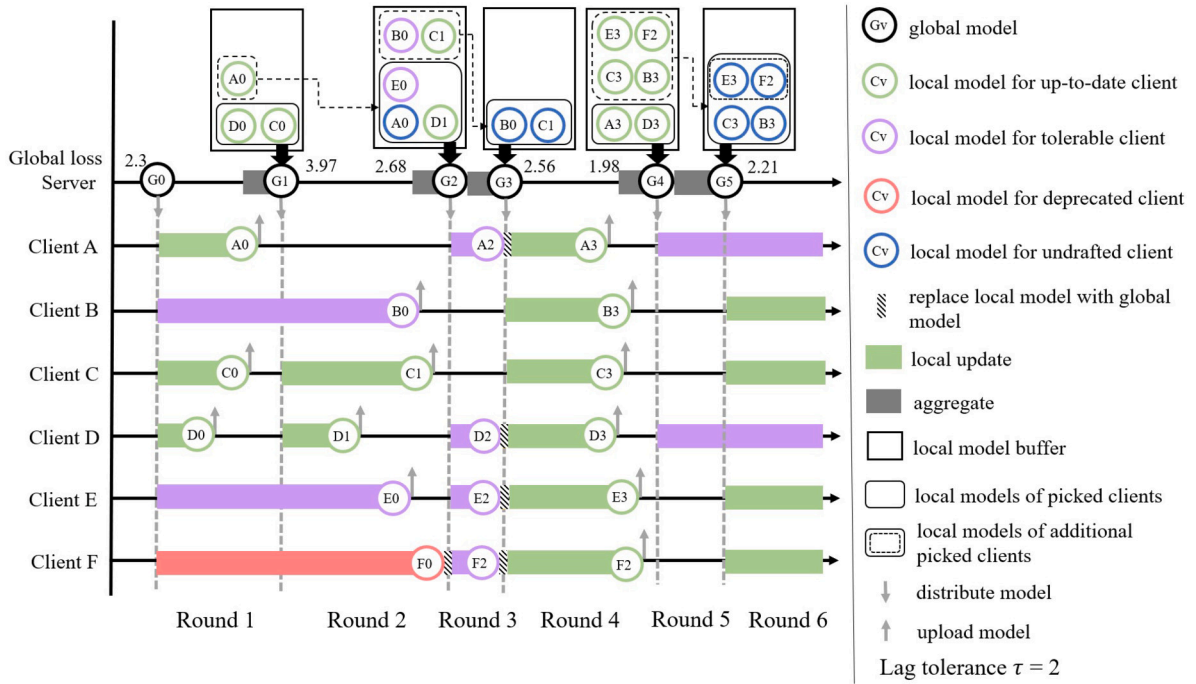


Fig. 4. The workflow of SASAFL.

the server in the previous round. (e.g. clients B and E in round two as shown in Fig. 4).

- Undrafted: Clients who uploaded a local model before the previous round but did not participate in the aggregation of the previous round. Consequently, they were still waiting to receive the global model (e.g. client A in rounds one and two as shown in Fig. 4).

Active clients include *deprecated*, *inferior-tolerable*, and *superior-tolerable* clients, defined as follows:

- Deprecated: Clients who still update the local model based on the outdated global model (e.g. client F in round two as shown in Fig. 4).
- Inferior-tolerable: Clients who update the local model based on the not-too-outdated global model. The *base global loss* was higher than the *latest global loss*. (e.g., clients B and E in rounds two as shown in Fig. 4).
- Superior-tolerable: Clients who update the local model based on the not-too-outdated global model, which has a lower base global loss than the latest global loss. (e.g., clients A and D in rounds five and six as shown in Fig. 4).

Here, the base global loss means that the loss of base global model of each client and the latest global loss means that the loss of latest global model.

Details of global model reception policy Among the six types of clients, waiting clients that participated in the previous round, and deprecated clients received and replaced the local model with the global model. Subsequently, they conducted a new local update. For the waiting clients who participated in the previous round, the local update can begin based on an global model trained from their local dataset. In other words, this approach allows for efficient local updates by a suitable global model for each participating client. Waiting clients who do not participate automatically participate in the next round, as explained later. Therefore, if they wait for only one round, they can begin a local update using a global model that trained from their local dataset.

Deprecated clients continue to update their local models using an excessively outdated global model. Consequently, participation in the

aggregation may decrease the accuracy of the global model. This can lead to a problem in that it reduces the local model quality for the clients receiving the global model. Therefore, the deprecated clients should receive the global model.

Inferior-tolerable clients also received a global model. Even they update the local model using a not-too-outdated global model, it needs to consider that they may become deprecated clients because of their slower computation speed compared with that of up-to-date clients. If they become deprecated clients, they are replaced by the latest global model. However, as previously mentioned, this may be replaced by a global model of lower quality. Therefore, if a client has tolerable staleness and can replace the base model to a global model with a lower global loss, it can improve the quality of local model.

In SASAFL, a deprecated and inferior-tolerable client initiates a new local update using the received global model; however, this client performs a smaller quota (local epoch) than before. For example, assume that the default local update quota (i.e., local epoch) is five. If the global model is received before the completion of the local update, the local update is performed with four local epochs, which is one less than the previous quota. A tolerable client is more likely to become a deprecated client because of its slower computation speed compared with an up-to-date client. That is, performing the same number of local updates whenever a client receive the global model can result in become stragglers. Consequently, the global model cannot be trained local datasets held by these clients. To prevent this problem, a deprecated and inferior-tolerance client performs a local update that is less than the previous local epoch.

Clients classified as superior tolerable had an acceptable level of staleness. These clients performed local updates using a global model with a lower base global loss than that of the latest global loss. That is, there is no need to receive a global model, because the quality of the local model can be guaranteed even without receiving the global model.

As shown in Fig. 4, one of the up-to-date clients is client D in round one. Client D received G_0 (i.e., the global model of version 0) and successfully uploaded the local model in round one. Client E received G_0 and uploaded the local model in round two with a staleness of 1 while the lag tolerance set to 2. Therefore, the client E considered it tolerable. Client A in round one received G_0 and successfully uploaded the local model in round one. However, it did not participate in round one, but in

round two. Therefore, it becomes an undrafted client in round one and two. These three clients participated in round two. Based on the reception policy for waiting clients, they received G2 and began a new local update based on the global model in round three.

Client F in round two was marked as a deprecated client because it failed to complete the local update in two consecutive rounds. This implies that the staleness of the local model of client F in round three was greater than the lag tolerance which was set to 2 in this example. Therefore, it needs to receive a global model. After receiving G2 in round three, client F started with the less amount of local update than round two. However, the global loss of G3 was reduced from 2.68 to 2.56, resulting in clients A, D, E, and F become inferior-tolerable clients. This indicates that, these clients also receive G3.

Finally, clients A and D in round six were considered superior-tolerable clients. They received G4. However, because the latest global loss increased from 1.98 to 2.21, they did not receive the global model.

Algorithm 1: Global model reception policy.

```

Input: local model version  $v^k$ , global model version  $r$ , latest global loss  $gl_r$ , base global loss  $gl_{r-1}$ , lag tolerance  $\tau$ , local epoch  $E$ 
/* active client */
1 if client  $k$  still updating local model then
    /* deprecated client */
    2 if  $r - v^k \geq \tau$  then
        3 receive global model  $w_r$ 
        4  $w^k \leftarrow w_r$ 
        5  $E^k \leftarrow \min(E^k - 1, \frac{E}{2})$ 
    /* inferior-tolerable client */
    6 else if  $gl_r < gl_{r-1}$  then
        7 receive global model  $w_r$ 
        8  $w^k \leftarrow w_r$ 
        9  $E^k \leftarrow \min(E^k - 1, \frac{E}{2})$ 
    /* superior-tolerable client */
    10 else
        11 not receive global model
    12 end
/* waiting client */
13 else if client  $k$  participated in the aggregation of the previous round then
    14 receive global model  $w_r$ 
    15  $w^k \leftarrow w_r$ 
16 end

```

Algorithm 1 describes the global model reception policy of the SASAFL. In the case of the deprecated and inferior-tolerable clients, they received a global model (lines 3 and 7) and adjusted their local epoch to $E^k - 1$ for start reduced local update (lines 5 and 9) where E^k denotes the quota of previous local update of client k . Note that, it is reduced until the minimum local epoch where underfitting will not occur. The minimum local epoch is hyperparameter, and we set it to half of the origin quota E empirically. In the case of superior-tolerable clients, they do not receive a global model (line 11). Among the waiting clients, those who participated in the previous round performed a new local update after receiving the global model (lines 14 and 15). Lag tolerance is a hyperparameter, and to identify the optimal value, we adjust τ from 1 to 10 in the experiments section.

4.2. Adjusting number of participating clients

Details of adjusting number of participating clients After aggregation, the server calculates the number of clients participating in the new round. The number of participating clients was adjusted based on the quality of the global model. This adjustment was made to address the problem of increased time required for one round when many clients participated each time. As aforementioned, in the SASAFL, the quality of the global model is defined as a loss. If the global loss after aggregation increased compared with the previous round, the SASAFL increases

the number of clients participating to improve the quality of the global model. This allowed the server to be trained using various local datasets. On the contrary, if the global loss of the updated global model was lower than that of the previous round, it indicated an improvement in quality of the global model and resulted in a reduction in the number of participating clients. If the FL process involved a larger number of participating clients, it could converge to high accuracy more quickly. Although this approach leads to increased latency, as the quality of the global model improves, the training can progress stably. Therefore, there was no necessity to increase the number of clients when the global loss decreased. If the global loss increased again when aggregation was performed with a reduced number of participating clients, the quality of the global model could be improved by increasing the number of participating clients. In summary, the SASAFL increases the number of participating clients as required based on an increase in global loss.

The number of participating clients increases or decreases proportionally to the rate of the global loss change. If the global loss increased more than that in the previous round, the number of clients in the next aggregation would increase at the same rate as the increase in the global loss. Currently, the maximum number of participating clients is limited by the total number of clients. On the contrary, if the global loss in the current round is lower than that in the previous round, the number of clients is reduced by the reduction ratio of the global loss. The minimum number of participating clients is a hyperparameter and is the same as the number of participating clients used in previous semi-asynchronous FL. The formula for adjusting the number of participating clients in SASAFL is as follows:

$$P_{r+1} = \begin{cases} \min(P_r \times \frac{gl_r}{gl_{r-1}}, K) & \text{if } gl_r > gl_{r-1}, \\ \max(P_r \times \frac{gl_r}{gl_{r-1}}, P_{min}) & \text{if } gl_r < gl_{r-1} \end{cases} \quad (1)$$

where P_r denotes the number of participating clients in round r , gl_r denotes the global loss of version r (i.e., round r), K is the total number of clients, and P_{min} is the minimum number of participating clients.

SASAFL assumes that the server performs aggregation and receives local models in parallel. The local models received during aggregation were collected in the buffer. When the next round began, all the local models stored in the buffer were used for aggregation. That is, clients with local models stored in the buffer automatically participated in the next round. As SASAFL does not strictly enforce the number of participating clients in the aggregation, it allows more clients participate than the quota which is calculated using Eq. (1). However, if there are more local models in a buffer than in a quota, the aggregation begins immediately with all the local models in the buffer.

As shown in Fig. 4, the global loss of G1 is 3.97, which is 1.72 times higher than G0. According to Eq. (1), three clients participated in the aggregation for G2, which is a 1.72 times increased quota from the aggregation for G1. Currently, the local model of undrafted client A from round one is utilised for the aggregation of round two. Therefore, the server waited until two clients other than Client A participated. Because clients D and E uploaded their local models in round two, the aggregation started immediately after receiving the local model of client E. During the aggregation for G2, the server received local models from clients B and C. Consequently, two local models were stored in the buffer at the end of round two. Because global loss of G2 decreased by 0.67 times compared with G1, the number of clients participating in round three became two, which was a 0.67 times decreased from the three clients participating in the round two. However, because there were two undrafted clients, the aggregation for G3 began again as soon as the aggregation for G2 completed. In the case of the G5 aggregation, the calculated quota was two. However, because four local models were received during the aggregation for G4, two additional clients (clients E and F) participated, and the aggregation process started immediately.

Algorithm 2: SASAFL.

Input: local model buffer B , minimum number of participating client P_{min} , global model version r , participating clients set $P(r)$, global model w , global loss gl_r

```

1 Server process:
2    $r = 0$ 
3    $P_r = P_{min}$ 
4    $B = \emptyset$ 
5   Listening thread:
6     Receive  $w^k$  from client  $k$ 
7     Push  $w^k$  in  $B$ 
8   Aggregation thread:
9     Distribute  $w_r$ ,  $gl_r$ , and  $r$  to all clients
10    Wait for  $len(B) \geq P_r$ 
11     $P(r) = B$ 
12     $B = \emptyset$ 
13    Refine  $P(r)$  according to Eq. (2)
14     $w_{r+1} = \text{Aggregate } P(r)$ 
15    Update  $P_{r+1}$  according to Eq. (1)
16     $r = r + 1$ 
17 Client process:
18   Listening thread:
19     Receive global model according to Alg. 1
20   Local update thread:
21   /* Whenever the global model received          */
22     LocalUpdate( $w^k$ ,  $E^k$ )
23     Send local model  $w^k$  to server

```

4.3. The SASAFL protocol

For poor convergence performance owing to the staleness of the local model, SASAFL applied the global model reception policy on the client and adjusted the number of participating client policies on the server. Algorithm 2 represents the workflow of the SASAFL.

When the FL process began, the server initialised the index of the round (i.e., global model version) to zero, the number of participating clients for the first round to P_{min} , and the local model buffer to an empty set (lines 2-4). When a new round starts, the server distributes the global model w_r , global loss gl_r , and index of round (i.e., global model version) r to all clients (line 9). Simultaneously, the listening thread is running to receive the local model from any clients (line 6 and 7). In the client process, the global model is received in a listening thread, while the local update is performed in a local update thread. These threads run in parallel, similar to a server. Whenever a client receives the global model, by global model reception policy, they also update their local epoch and replace local model as needed (line 19) and then begin the local update (line 22). If a local update is successfully completed, the local model is sent to the server (line 23). This locally sent model is then received in the listening thread of the server (lines 6 and 7). The server waits until the number of local models in the buffer B meets the threshold P_r (line 10). This indicates that the server waits until at least P_r clients can participate in aggregation.

Before the aggregation, the server refines local models in the buffer (line 13). The formula [17,18] for refining the local model is as follows:

$$w^k = \zeta^{s^k} w^k + (1 - \zeta^{s^k}) w_r \quad (2)$$

where ζ denotes the decay coefficient, and s^k denotes the staleness (i.e., the difference in version between the local and global models) of the client k .

The purpose of refining the local model is to reduce the pollution caused by the staleness of the local model in the global model. Although SASAFL has attempted to improve the quality of the local model through the global model reception policy, the presence of a stale local model prevents the attainment of an optimally updated global model. Therefore, a penalty must be imposed on the local model based on the degree of staleness.

After the server refined the local models of the participating clients, the aggregation for the global model is begun using the same formula as in [2] (line 14). The aggregated result becomes the global model for the next round. The number of participants in the next round was determined using to Eq. (1) when the global model was updated (line 15). This process completes one round (line 16).

5. Experiment and evaluation**5.1. Experiment setup**

Testbed. To validate the effectiveness of SASAFL, we developed an FL testbed platform on a Nurion [22] CPU cluster in Korea Institute of Science and Technology Information (KISTI). Servers and clients were deployed on a KNL CPU cluster, where each node had 68 cores. This testbed platform was implemented on PyTorch 1.12 [23], and communication between the server and client was implemented using the distributed communication package in PyTorch [24], with a built-in backend gloo for CPU training.

For system heterogeneity, each client executes an FL process with one or more cores. Clients were assigned to one of the five categories. Therefore, when K is 30, six clients were assigned to each category, and when K is 100, twenty clients were assigned to each category. When K is 30, the numbers of cores assigned to each category were 1, 2, 3, 4, and 8, and when K is 100, they were the same, except that five cores were used instead of four.

For statistical heterogeneity, we prepared a non-IID configuration with different distributions and numbers of data points for each client. The quantity of data for each client was randomly determined, following a Gaussian distribution. The divided datasets were sequentially allocated based on the client index. Therefore, each client had one or two labels.

Benchmarks. To evaluate the SASAFL, we implemented the following semi-asynchronous benchmarks in our testbed, and compared the performance of each method.

- **Basic semi-asynchronous FL (BSA):** It is identical to the SASAFL except the global model reception policy and adjusting number of participating client algorithm. In other words, even if the number of local models received during aggregation is greater than the determined number of local models, the server only uses fixed number of local models for aggregation. In common with SASAFL, the aggregation and the local model reception in server perform in parallel. Additionally, a penalty is applied when aggregating the local models, according to Eq. (2).
- **SAFA [15]:** It is state-of-the-art solutions for the semi-asynchronous FL considering accuracy and latency. Among the clients, the deprecated and up-to-date clients are synchronised with the server. And there are crashed clients who cannot participate in training each round. Considering the latency, the maximum waiting time for one round was limited to the slowest client among all clients or a user-defined deadline.

Model and datasets. In this experiment, we used CNN models identical to those in [2] and three benchmark datasets, MNIST [12], FashionMNIST [25], and CIFAR-10 [26]. The models employed for training the MNIST and FashionMNIST datasets were identical.

Training parameters. The following parameters were used in the experiments: We set the total number of clients K to 30 and 100, and the (minimum) participating ratio C of the clients to 0.1 [2], 0.3 [8], and 0.5. In algorithm 2, P_{min} is calculated by multiplying C and K . The learning rate was 0.01 for MNIST, 0.001 for FashionMNIST, and CIFAR-10 showing the highest performance for all methods in our experimental setting. The number of local epochs E was set to five, and the minimum local epoch was set to half of E , which is threshold at which the local model avoids underfitting. The minimum local epoch is

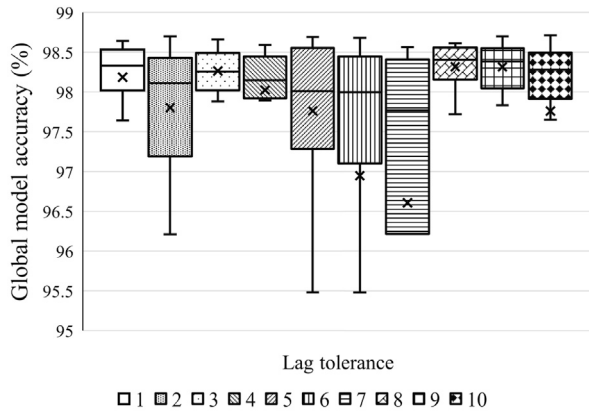


Fig. 5. Accuracy of global model according to lag tolerance changes from 1 to 10 on SASAFL.

a specific hyperparameter in SASAFL. If user wants that communication overhead should be low, it should be set close to the default local epoch for avoid frequently receiving global model. Otherwise, it is better to set it as small as possible in order to receive global model frequently. The batch size that the clients used for the local updates was set to 32. The number of rounds constituting the FL process differed for each dataset, and the experiment was conducted until the global model converged. The number of rounds was 500 for both MNIST and FashionMNIST. For CIFAR-10, 3000 rounds were used. The decay coefficient ζ in Eq. (2) was set to 0.9. All the experiments were performed with an average of ten repetitions per setting.

Metrics. To evaluate SASAFL, we used metrics defined as follows:

- **Accuracy:** The highest test accuracy was achieved during the FL process. The purpose was to evaluate the accuracy of the SASAFL.
- **Round-to-accuracy [27]:** Number of rounds required to reach the target accuracy. The purpose was to evaluate communication efficiency which is related to latency performances.
- **Time-to-accuracy [27]:** Elapsed time required to reach the target accuracy in minutes. The purpose was to evaluate the speed efficiency which is also related to the latency performances.

5.2. Experiment results

Lag tolerance SASAFL and SAFA require lag tolerance values. In the case of the SAFA, because the authors set the lag tolerance to five which could not a high communication cost and loss of the global model, we also set the lag tolerance of the SAFA to five. For the SASAFL, the lag tolerance was varied from 1 to 10 to determine the optimal value.

Fig. 5 shows that the accuracy of the global model has a large variation and low accuracy when the lag tolerances are 5, 6, and 7. If the lag tolerance is small value (i.e., 1, 2, 3, or 4), the clients can be periodically synchronised with the server. On the other hand, if the value is large (i.e., 8, 9, or 10), there is a higher chance of engaging in local training using a more recent and higher-quality global model. This leads to lower variations in the accuracy of the global model and the likelihood of achieving a higher accuracy.

Based on these observations, it is crucial for the overall training to set a lag tolerance that allows minimal latency while ensuring high accuracy. This approach also contributes to maintaining stability in the overall training process by minimising variation in the accuracy of global model. For optimal training performance, the lag tolerance must be set as follows: 1) allowing a high lag tolerance to minimise the communication overhead, 2) ensuring high accuracy through periodic synchronisation with the server, and 3) minimising the variation in the accuracy of the global model for training stability. Therefore, we set the lag tolerance to nine which shows the highest accuracy of the global model. However, to determine the optimal lag tolerance for user, it is

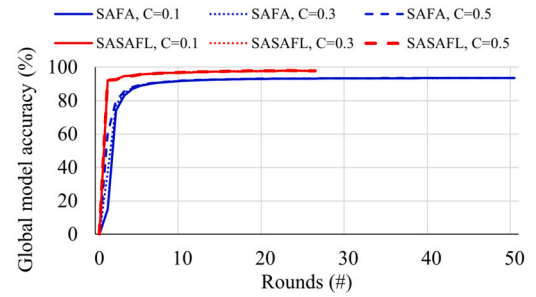


Fig. 6. Training curve on MNIST comparison with SAFA (blue) and SASAFL (red) when K set to 100. The solid, dotted, and dashed line correspond to C set to 0.1, 0.3, and 0.5, respectively.

Table 1

Wall clock time comparison between SAFA and SASAFL on MNIST when K set to 100. Each value represents the wall clock time in minutes required to complete 25 rounds.

Method	SAFA	SASAFL
C = 0.1	21.21	16.48
C = 0.3	21.26	16.3
C = 0.5	21.21	16.8

better to manually search from 1 to 10 according to their model and datasets.

Training curve In the comparison between SAFA and SASAFL, the experimental settings are different with described above. First, we use only MNIST datasets and the learning rate was configured at 0.001, while the batch size was established at 40. The total number of clients used in the evaluation was 100. In addition, in the settings in section 5.1, both the distribution and size of local datasets were set to be heterogeneous, but in comparison with SAFA, the only sizes of local datasets were set to randomly follow Gaussian distribution. The reason for changing the settings is to make it the same as the experimental setting introduced by SAFA.

Fig. 6 shows the training curves of SAFA and SASAFL using MNIST. SASAFL reached an accuracy of over 90% from the beginning of training regardless of the participating ratio C . In a setting where C set to 0.3 and 0.5, SASAFL converged to the target accuracy of 98% in 22 rounds. In other settings (i.e., C set to 0.1), convergence to target accuracy was achieved in 25 rounds. On the other hand, SAFA reached 93%, which is lower than the target accuracy in all C settings.

In particular, it can be seen that the higher the participating client ratio, the convergence to higher accuracy was achieved, albeit slightly. The reasons why SASAFL was able to show higher performance than SAFA are as follows. Unlike SAFA, SASAFL determines the number of participating clients based on global loss. Given that global loss might often increase during the initial phases of FL process, improving the quality of the global model involves maximising client participation as much as possible.

Table 1 compares the wall clock time of SAFA and SASAFL to complete 25 rounds on MNIST when K set to 100. SASAFL showed shorter wall clock time than SAFA in all C settings. Unlike SASAFL, SAFA considers crashed clients, and because all clients except crashed clients participate in each round, the time required for one round is long. However, SASAFL can reduce the time required for each round because the number of participating clients increases only when the global loss increases.

Furthermore, SAFA converged to reasonable accuracy only when size of the local dataset was heterogeneous. It was also observed that the global model was not trained at when the distribution of local dataset was heterogeneous. Therefore, in future experimental results, the performance was evaluated only with BSA in our heterogeneity settings, excluding SAFA.

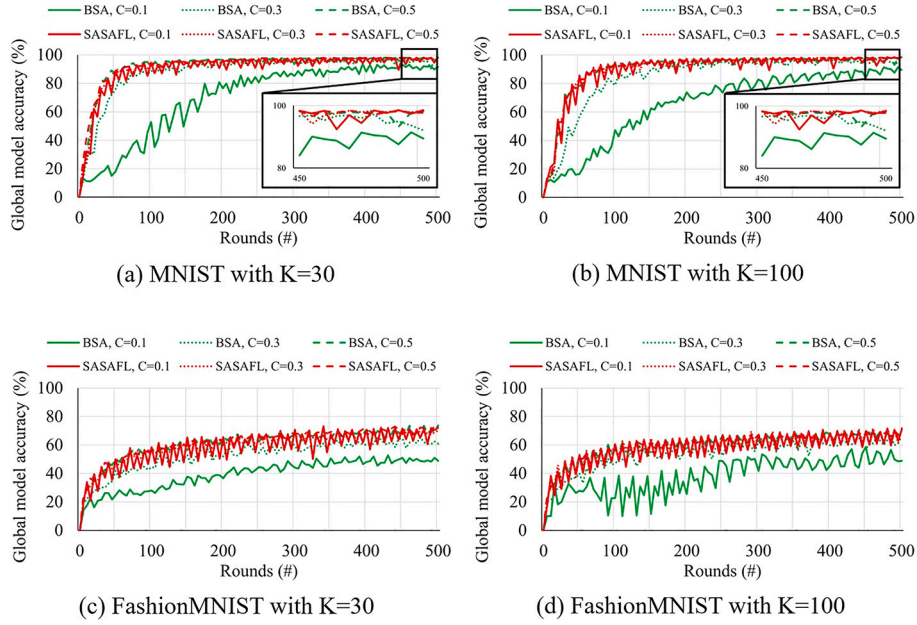


Fig. 7. Training curve on MNIST (the above two) and FashionMNIST (the below two) comparison with BSA (green) and SASAFL (red). The solid, dotted, and dashed lines are corresponding to C set to 0.1, 0.3, and 0.5, respectively. Left and right correspond to K set to 30 and 100. (All graphs were floated every 5 rounds.)

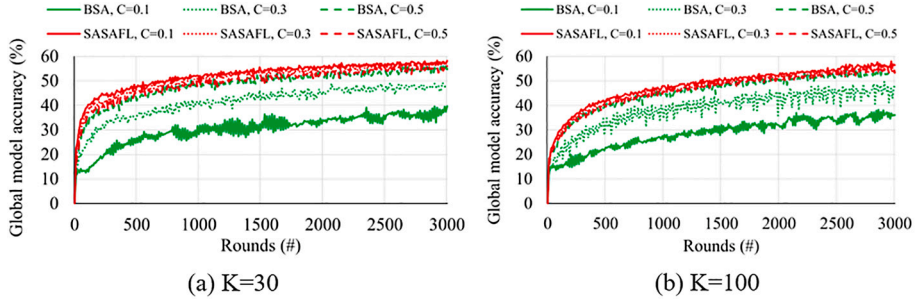


Fig. 8. Training curve on CIFAR10 comparison with BSA (green) and SASAFL (red). The solid line, dotted line, and dashed line denotes C set to 0.1, 0.3, and 0.5, respectively. Left and Right correspond to K set to 30 and 100 each. (All graphs floats every ten rounds.)

Fig. 7 shows the training curve for the benchmark on BSA, and SASAFL with MNIST ((a) and (b)) and FashionMNIST ((c) and (d)) when the total number of clients is 30 (left) and 100 (right). All four graphs showed similar trends in the training curves. SASAFL outperformed the accuracy performance of BSA in all settings except for the setting BSA with C set to 0.5 whose accuracy was 98.27% (Table 2).

Note that the SASAFL converged to a high accuracy even if the fraction of clients participating in aggregation among all clients is small (i.e., C set to 0.1). This is because the policy of adjusting the number of participating clients in SASAFL does not strictly limit the number of participating clients, allowing for the rapid aggregation of stored local models in the buffer. However, the fluctuation in the training curve was greater than in settings with a high fraction of participating clients. This training curve is the same in BSA. This is because, as the participating ratio decreases, the frequency with which the global model was updated increased, thereby increasing the staleness of the clients' local models. Fig. 7-(c) and (d) demonstrate that as more than 10% of clients participate (i.e., C set to 0.3 and 0.5), the training curve converges steadily.

Meanwhile, SASAFL's global model reception policy has an advantage in that the fluctuation in accuracy of SASAFL was lower than that of BSA, except Fig. 7-(c). In other words, it can be observed that receiving a global model with high quality by considering global loss increases the stability of training.

Fig. 8 shows the training curve for CIFAR-10 which is a more complex dataset than the MNIST and FashionMNIST. SASAFL outperformed

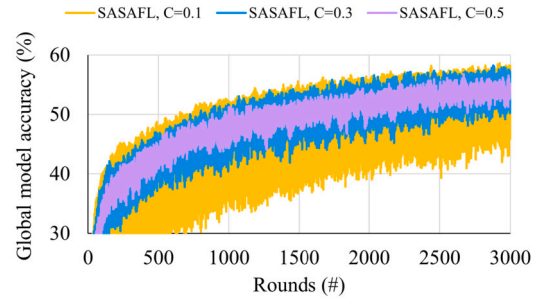


Fig. 9. Training curve on CIFAR10 of SASAFL when K set to 30. The yellow line, blue line, and purple line denotes C set to 0.1, 0.3, and 0.5, respectively. (This graph was floated every one round.)

BSA in terms of accuracy under all settings. In particular, the lowest accuracy among the three SASAFL settings and the highest accuracy among the three BSA settings were the same. It is worth noting that the highest accuracy when C set to 0.1 of SASAFL, which has the lowest ratio of participating clients. However, Fig. 9 shows that the training curve of C set to 0.1 is more unstable than C set to 0.3 and 0.5. This reason is the same as that explained above, and it can be observed once again that the training curve becomes more stable as more clients participate in aggregation. However, there is no significant difference in the accuracy of SASAFL even when the participating ratio changes.

Table 2

Performance results of accuracy, round-to-accuracy, and time-to-accuracy on three benchmark datasets which are MNIST, FashionMNIST, and CIFAR-10 comparison to BSA with C set to 0.1, 0.3, and 0.5, and K set to 30.

Dataset	MNIST		FashionMNIST		CIFAR-10	
Method	BSA	SASAFL	BSA	SASAFL	BSA	SASAFL
C	0.1					
Accuracy	93.78	98.23	54.72	74.09	42.5	58.59
Round-to-accuracy	-	116	-	248	-	1,490
Time-to-accuracy	-	53.86	-	105.24	-	1,104.42
C	0.3					
Accuracy	97.14	98.07	66	71.59	51.08	58.25
Round-to-accuracy	211	132	-	420	-	1,728
Time-to-accuracy	101.44	66.71	-	178.8	-	1,281.03
C	0.5					
Accuracy	98.27	98.24	73.834	73.94	56.82	56.77
Round-to-accuracy	96	131	376	283	2,348	2,140
Time-to-accuracy	60.98	88.77	185.34	142.21	1,809.73	1,680.49

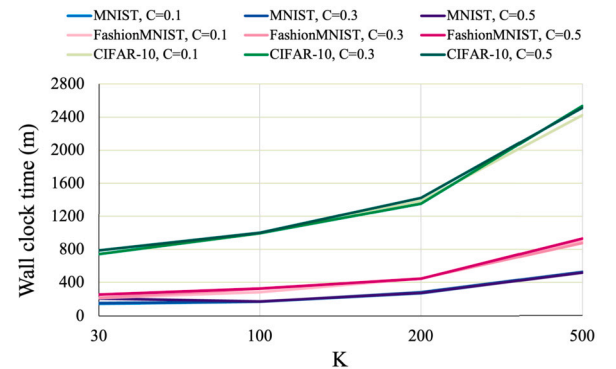
Table 3

Performance results of accuracy, round-to-accuracy, and time-to-accuracy on three benchmark datasets which are MNIST, FashionMNIST, and CIFAR-10 comparison to BSA with C set to 0.1, 0.3, and 0.5, and K set to 100.

Dataset	MNIST		FashionMNIST		CIFAR-10	
Method	BSA	SASAFL	BSA	SASAFL	BSA	SASAFL
C	0.1					
Accuracy	92.47	98.56	65.4	71.74	38.26	57.95
Round-to-accuracy	-	119	-	382	-	2,418
Time-to-accuracy	-	65.99	-	214.04	-	2,384.51
C	0.3					
Accuracy	97.31	98.52	66.08	71.87	49.65	56.41
Round-to-accuracy	244	105	-	416	-	2,522
Time-to-accuracy	131.37	58.5	-	268.08	-	2,356.59
C	0.5					
Accuracy	98.27	98.36	69.99	70.57	55	56.31
Round-to-accuracy	131	133	-	416	2,670	2,563
Time-to-accuracy	82.65	74.81	-	371.15	2,287.45	2,354.96

Accuracy and latency performance Table 2 and 3 compare the accuracy and latency performance of BSA and SASAFL when the total number of clients is 30 and 100, respectively. To compare latency performance, two metrics which are round-to-accuracy and time-to-accuracy are measured. The target accuracy was set to 95% for MNIST, 70% for FashionMNIST, and 55% for CIFAR-10. In all cases, except for the setting in which the participating client ratio was 50%, SASAFL outperformed the accuracy and latency of BSA. When the ratio of participating clients was small, BSA failed to converge on the target accuracy, whereas SASAFL reached all target accuracies. However, in a setting where half of all clients participated (i.e., C set to 0.5) on MNIST when K set to 30, the accuracy, round-to-accuracy, and time-to-accuracy performance of BSA was higher than SASAFL although the accuracy performance between BSA and SASAFL is almost same. We analyzed that, if many clients can participate, the simpler dataset like MNIST, the more overhead the policies applied to SASAFL become in training. However, in another dataset setting where half of the clients participate in aggregation, SASAFL showed similar performance compared to BSA in terms of accuracy and higher performance in terms of latency except when C set to 0.5 and K set to 100 on CIFAR-10.

Among all settings, when the participating ratio is 10% and total number of client is 100 on CIFAR-10, the accuracy performance of SASAFL was 19.69%p higher than that of BSA. Additionally, among settings where both BSA and SASAFL reached target accuracy, in the MNIST setting, where 30% of all 30 clients participated, SASAFL's

**Fig. 10.** Wall clock time of SASAFL according to the same setting as in Table 4.

round-to-accuracy and time-to-accuracy were 2.32 times and 2.24 times higher than BSA's, respectively. Again, the participating client ratio in SASAFL is intended to determine the minimum number of clients that can participate in aggregation, so, more clients can participate than determined number of participating clients. Based on these observations, SASAFL increased the stability of training through a global model reception policy and improved the accuracy and latency performance of training by adjusting number of participating clients policy.

Table 4

Wall clock time of SASAFL as the total number of clients increases for three benchmark datasets and the participating ratio. SASAFL was run for 300 rounds on MNIST, 500 rounds on FashionMNIST, and 1000 rounds on CIFAR-10.

Dataset	MNIST			FashionMNIST			CIFAR-10		
C	0.1	0.3	0.5	0.1	0.3	0.5	0.1	0.3	0.5
K=30	139	151	204	212	213	251	741	742	787
K=100	166	167	168	280	322	326	989	993	1,000
K=200	267	280	270	446	448	444	1,384	1,351	1,421
K=500	521	527	518	894	876	930	2,423	2,533	2,512

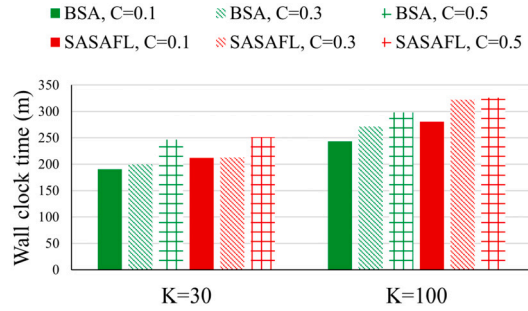


Fig. 11. Comparison of wall clock time between BSA and SASAFL on FashionMNIST during 500 rounds.

Table 4 and Fig. 10 depicts the wall clock time of SASAFL in relation to the augmented total number of clients for 300 rounds for MNIST, 500 rounds for FashionMNIST, and 1,000 rounds for CIFAR-10. For all datasets, an increase in both the participating ratio (C) and the total number of clients (K) leads to a rise in the wall clock time for the same workload. It is due to the concurrent rise in the communication overhead with the increase of number of clients. Remarkably, the accuracy of the global model remains consistent across various settings, similar to the results observed in Table 3 and Table 4. Thus, SASAFL demonstrates significant scalability, though accompanied by an inevitable increase in communication overhead as the scale increases.

Limitation of SASAFL However, the SASAFL has limitation in terms of latency. According to Eq. (1), the number of clients participating in aggregation can be increased by the total number of clients. Similar to synchronous FL, the wall clock time of this round is bound to the slowest client among all clients. If the global model is trained unstable, the overall training speed may slow down because of this limitation. For example, Fig. 7-(c) shows a relatively unstable training curve, therefore it can be observed that the wall clock time of the SASAFL was high under this setting, as shown in Fig. 11. SASAFL had the longer wall clock time than BSA, except for the setting in which the total number of clients was 100 and the participating client ratio was 50%. However, this limitation can be solved by setting a maximum deadline within one round, as in SAFA.

6. Conclusion and future works

In this study, we proposed the SASAFL to solve staleness issues in semi-asynchronous FL. A global model reception policy that considers the global loss and lag tolerance can stabilise training by increasing the quality of the local model, which is closely related to the performance of the global model. In addition, the efficiency of training was increased by allowing clients who failed to participate in the previous round to participate in the next round, along with adjusting the number of participating clients according to the changes in global loss. We evaluated the performance of the SASAFL under various conditions and datasets. The SASAFL achieved up to 19.69% higher accuracy and in terms of latency, it achieved 2.32 times higher round-to-accuracy and 2.24 times higher time-to-accuracy than baseline. There is a latency limitation in that the number of participating clients can be equal to the total number

of clients; however, this can be resolved by setting a deadline for each round. Therefore, our future work will involve adjusting the deadline to ensure accuracy and latency.

CRedit authorship contribution statement

Miri Yu: Writing – original draft, Methodology, Investigation, Conceptualization. **Jiheon Choi:** Writing – original draft, Investigation, Conceptualization. **Jaehyun Lee:** Software, Data curation. **Sangyoon Oh:** Writing – review & editing, Validation, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Sangyoon Oh reports financial support was provided by Institute of Information and Communications Technology Planning and Evaluation (IITP). Miri Yu reports equipment, drugs, or supplies was provided by Korea Institute of Science and Technology Information (KISTI). Sangyoon Oh reports a relationship with National Science Foundation of Korea (NRF-Korea) that includes: consulting or advisory and funding grants. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was jointly supported by the Korea Institute of Science and Technology Information (KISTI) (KSC2022-CRE-0406), NRF-Korea grant funded by the Korea government (MSIT) (RS-2023-00283799), and Artificial Intelligence Convergence Innovation Human Resources Development by the Institute of Information and Communications Technology Planning and Evaluation (IITP-2023-No.RS-2023-00255968). The authors would like to thank Editage (www.editage.co.kr) for English language editing.

References

- [1] P. Kairouz, H.B. McMahan, et al., Advances and open problems in federated learning, *Found. Trends Mach. Learn.* 14 (2021) 1–210.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: *Artificial Intelligence and Statistics*, PMLR, 2017, pp. 1273–1282.
- [3] T. Nishio, R. Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: *ICC 2019-2019 IEEE International Conference on Communications (ICC)*, IEEE, 2019, pp. 1–7.
- [4] Z. Chai, A. Ali, S. Zawad, et al., Tifi: a tier-based federated learning system, in: *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*, 2020, pp. 125–136.
- [5] T. Li, A.K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, *Proc. Mach. Learn. Syst.* 2 (2020) 429–450.
- [6] J. Park, D. Yoon, S. Yeo, S. Oh, Amble: adjusting mini-batch and local epoch for federated learning with heterogeneous devices, *J. Parallel Distrib. Comput.* 170 (2022) 13–23.
- [7] F. Lai, X. Zhu, H.V. Madhyastha, M. Chowdhury, Oort: efficient federated learning via guided participant selection, in: *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 19–35.

- [8] Y.J. Cho, J. Wang, G. Joshi, Towards understanding biased client selection in federated learning, in: International Conference on Artificial Intelligence and Statistics, 2022, pp. 10351–10375.
- [9] J. Zhang, X. Cheng, et al., Fedada: fast-convergent adaptive federated learning in heterogeneous mobile edge computing environment, World Wide Web 25 (2022) 1971–1998.
- [10] C. Xie, S. Koyejo, I. Gupta, Asynchronous federated optimization, arXiv preprint, arXiv:1903.03934, 2019.
- [11] T. Zhang, L. Gao, S. Lee, M. Zhang, S. Avestimehr, Timelyfl: heterogeneity-aware asynchronous federated learning with adaptive partial training, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2023, pp. 5064–5073.
- [12] L. Deng, The mnist database of handwritten digit images for machine learning research [best of the web], IEEE Signal Process. Mag. 29 (2012) 141–142.
- [13] L. Yu, X. Sun, R. Albelaihi, C. Yi, Latency-aware semi-synchronous client selection and model aggregation for wireless federated learning, Future Internet 15 (2023) 352.
- [14] F. Liang, Q. Yang, R. Liu, J. Wang, K. Sato, J. Guo, Semi-synchronous federated learning protocol with dynamic aggregation in Internet of vehicles, IEEE Trans. Veh. Technol. 71 (2022) 4677–4691.
- [15] W. Wu, L. He, W. Lin, R. Mao, C. Maple, S. Jarvis, Safa: a semi-asynchronous protocol for fast federated learning with low overhead, IEEE Trans. Comput. 70 (2020) 655–668.
- [16] Q. Ma, Y. Xu, H. Xu, Z. Jiang, L. Huang, H. Huang, Fedsta: a semi-asynchronous federated learning mechanism in heterogeneous edge computing, IEEE J. Sel. Areas Commun. 39 (2021) 3654–3672.
- [17] J. Liu, H. Xu, L. Wang, Y. Xu, C. Qian, J. Huang, H. Huang, Adaptive asynchronous federated learning in resource-constrained edge computing, IEEE Trans. Mob. Comput. 22 (2023) 674–690.
- [18] J. Yu, R. Zhou, C. Chen, B. Li, F. Dong, Asfl: adaptive semi-asynchronous federated learning for balancing model accuracy and total latency in mobile edge networks, in: Proceedings of the 52nd International Conference on Parallel Processing, Association for Computing Machinery, 2023, pp. 443–451.
- [19] Y. Zhang, D. Liu, M. Duan, L. Li, X. Chen, A. Ren, Y. Tan, C. Wang, Fedmds: an efficient model discrepancy-aware semi-asynchronous clustered federated learning framework, IEEE Trans. Parallel Distrib. Syst. 34 (2023) 1007–1019.
- [20] B. Fan, X. Su, S. Tarkoma, P. Hui, Behave differently when clustering: a semi-asynchronous federated learning approach for iot, ACM Trans. Sens. Netw. 20 (2024) 1–28.
- [21] J. Liu, J. Jia, T. Che, C. Huo, J. Ren, Y. Zhou, H. Dai, D. Dou, Fedasmur: efficient asynchronous federated learning with dynamic staleness-aware model update, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2024, pp. 13900–13908.
- [22] Nurion (Online), <https://www.ksc.re.kr/eng/resources/nurion>. (Accessed 5 December 2023), 2023.
- [23] Pytorch (Online), <https://pytorch.org/>. (Accessed 5 December 2023), 2023.
- [24] Distributed communication package - torch.distributed (Online), <https://pytorch.org/docs/stable/distributed.html>. (Accessed 5 December 2023), 2023.
- [25] H. Xiao, K. Rasul, R. Vollgraf, Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, arXiv preprint, arXiv:1708.07747, 2017.
- [26] A. Krizhevsky, G. Hinton, et al., Learning Multiple Layers of Features from Tiny Images, 2009.
- [27] J. Shin, Y. Li, Y. Liu, S.-J. Lee, Fedbalancer: data and pace control for efficient federated learning on heterogeneous clients, in: Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services, 2022, pp. 436–449.



Miri Yu received the master degree from the Department of Artificial Intelligence at the Ajou University. Her research interests include federated learning and distributed computing in heterogeneous environments.



Jiheon Choi received his B.S in the Department of Software Engineering from the Ajou University in 2023. Currently, he is a Master student of the Department of Artificial Intelligence at the Ajou University. His research interests include offensive security, cloud computing security, adapt security suited for MLOps and distributed systems.



Jaehyun Lee received his B.S in the Department of Software Engineering from the Ajou University in 2023. Currently, he is a Master student of the Department of Artificial Intelligence at the Ajou University. His research interests include big data, cloud computing, and distributed systems.



Sangyoon Oh received the PhD degree from the Computer Science Department, Indiana University, Bloomington, Indiana. He is currently a professor of the School of Information and Computer Engineering, Ajou University, South Korea. Before joining Ajou University, he worked for SK Telecom, South Korea. His main research interests include high-performance computing, distributed deep learning, and graph data.